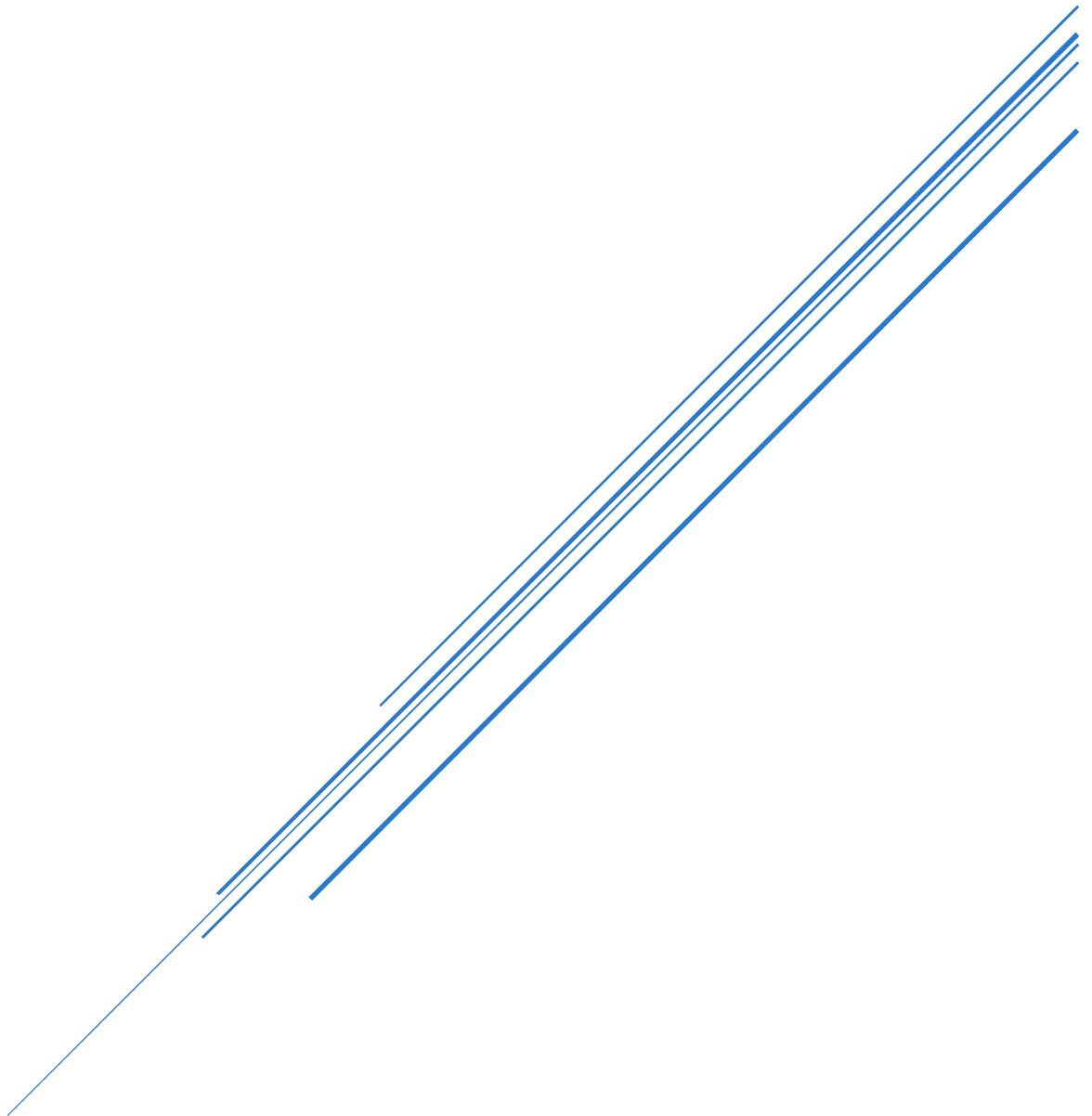


ACTIVIDAD EVALUABLE RA1

Óscar Martino



2ºDAM GETAFE

Sumario

Introducción.....	3
Estructura de la aplicación.....	3
Ciclo completo del proceso.....	4
Justificación del formato elegido para la exportación.....	9
Pruebas (videos).....	9

Introducción

Escenario:

Voy a firmar un contrato para trabajar en una entidad bancaria como programador, y tengo que desarrollar una aplicación Java que gestione una cuenta bancaria, conserve sus datos entre ejecuciones y permita exportar sus movimientos

La aplicación deberá permitir:

- Crear una cuenta con un cliente asociado.
- Realizar movimientos: ingresos y retiradas de dinero.
- Consultar el saldo y el listado de movimientos.
- Guardar en disco los datos de la cuenta y recuperarlos en la siguiente ejecución.
- Exportar los movimientos a un formato legible.

Estructura de la aplicación

Descripción breve de las clases principales, sus relaciones y los atributos más importantes

El programa está formada por cuatro clases: Cliente, Cuenta, Movimiento y Main

Cliente, Movimiento y Cuenta implementan Serializable para la persistencia de sus objetos, para luego guardarlos en un archivo y que “no desaparezcan”

La clase Cliente representa al propietario de la cuenta bancaria, guardando su nombre, dni y teléfono; con su constructor y métodos get para obtener los datos. Y el método toString() para mostrar los datos del cliente

La clase Movimiento representa cada operación que se hace en la cuenta, se guarda la fecha, el tipo, el concepto, el importe y el saldo restante. Cada vez que se crea un movimiento se obtiene la fecha con el LocalDateTime.now(). Y el método toString() para mostrar la fecha, tipo, concepto, importe y saldo restante del movimiento

La clase Cuenta representa la cuenta bancaria que se gestiona, se guarda el cliente, el saldo disponible y los movimientos realizados (ArrayList), la cuenta se crea con un saldo de 0 euros y la lista de movimientos vacía. Con sus métodos de ingresar() para “meter dinero” y se crea un nuevo movimiento, retirar() para “sacar dinero” y se crea un nuevo movimiento (se ve/comprueba antes si el importe es válido y si hay dinero suficiente), mostrarInformacion() para mostrar los datos del cliente, el saldo restante y todos los movimientos, getSaldo() para ver el dinero que “queda” en la cuenta, getMovimientos() para ver la lista/colección de movimientos (ver todos los movimientos)

Y la clase Main que se encarga del funcionamiento del programa con sus métodos necesarios para trabajar con los ficheros, mostrar el menú, realizar operaciones y exportar los movimientos

Ciclo completo del proceso

Voy a incluir video de la ejecución del programa

```
private static final String CARPETA_DATOS = "datos"; 1 usage
private static final String FICHERO_CUENTA = "datos/cuenta.dat"; 2 usages
private static final String CARPETA_EXPORTACION = "exportacion"; 1 usage
private static final String FICHERO_CSV = "exportacion/movimientos.csv"; 2 usages
/**

public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in); SE CREA EL SCANNER
    try {
        crearDirectorios(); MIRA A VER SI EXISTEN LOS DIRECTORIOS Y SI NO LOS CREA
        Cuenta cuenta = cargarCuenta(); CARGA LA CUENTA, SI NO EXISTE DEVUELVE NULL
        if (cuenta == null) { SI NO EXISTE
            System.out.println("no existe una cuenta guardada");
            cuenta = crearCuenta(teclado); CREA UNA NUEVA
        } else { SI YA EXISTE SIGUE CON EL PROCESO/PROGRAMA
            System.out.println("cuenta recuperada correctamente");
        }
        mostrarMenu(teclado, cuenta); SE MUESTRA EL MENU CON LAS DIFERENTES FUNCIONALIDADES DEL PROGRAMA
        guardarCuenta(cuenta); SE GUARDA LA CUENTA
        System.out.println("\ncuenta guardada correctamente");
    } catch (IOException e) {
        System.out.println("error con los ficheros: " + e.getMessage());
    } catch (Exception e) { CONTROL DE EXCEPCIONES (TRY CATCH)
        System.out.println("Error: " + e.getMessage());
    } finally {
        teclado.close(); CERRAR EL SCANNER
    }
}
```

Al iniciar el programa se ejecuta el metodo crearDirectorios(), qu comprueba si existen las carpetas de datos y exportacion. Si no existen, se crean automáticamente, la carpeta de datos para guardar la cuenta y la de exportación para guardar el fichero CSV con todos los movimientos realizados

```
public static void crearDirectorios() throws IOException { 1 usage
    File datos = new File(CARPETA_DATOS); CREA REFERENCIA DE LA CARPETA datos
    if (!datos.exists()) { COMPRUEBA SI LA CARPETA datos EXISTE
        if (!datos.mkdirs()) { COMPRUEBA SI SE PUEDO CREAR LA CARPETA datos
            throw new IOException("No se pudo crear la carpeta datos");
        }
    }
    File exportacion = new File(CARPETA_EXPORTACION); CREA REFERENCIA DE LA CARPETA exportacion
    if (!exportacion.exists()) { COMPRUEBA SI exportacion EXISTE
        if (!exportacion.mkdirs()) { COMPRUEBA SI SE PUEDO CREAR exportacion
            throw new IOException("no se pudo crear la carpeta exportacion");
        }
    }
}
```

No lo he podido poner en la captura, pero comprueba si se ha podido crear la carpeta cuando ve que no existe (if dentro de if) y en ese momento intenta crear la carpeta en caso de no existir, si consigue crearla devuelve true y si no devuelve false (y el mensaje de error)

```
public static Cuenta cargarCuenta() throws IOException, ClassNotFoundException { 1usage
    File fichero = new File(FICHERO_CUENTA); crea una referencia a la ubicación del archivo
    if (!fichero.exists()) { pero no crea el archivo
        return null; COMPRUEBA SI EL ARCHIVO EXISTE, SI NO, O DEVUELVE NULL
    }
    ObjectInputStream entrada = new ObjectInputStream(new FileInputStream(fichero));
    Cuenta cuenta = (Cuenta) entrada.readObject(); LEE EL OBJETO QUE ESTA GUARDADO EN EL
    entrada.close(); ARCHIVO
    return cuenta; DEVUELVE LA CUENTA
}
```

ObjectInputStream entrada = new ObjectInputStream(new FileInputStream(fichero));
Esto primero abre el archivo para poder leerlo, después permite leer los objetos Java que fueron guardados antes
Cuenta cuenta = (Cuenta) entrada.readObject(); como readObject() devuelve un Object, se pone el (Cuenta) antes para decir que es un objeto de Cuenta

```
/**
 * Crea una cuenta nueva solicitando los datos del cliente.
 *
 * @param teclado scanner utilizado para leer datos
 * @return cuenta nueva
 */
public static Cuenta crearCuenta(Scanner teclado) { 1usage
    System.out.println("crear cuenta:");
    System.out.print("Nombre: ");
    String nombre = teclado.nextLine();
    System.out.print("DNI: ");
    String dni = teclado.nextLine();
    System.out.print("Teléfono: ");
    String telefono = teclado.nextLine();
    Cliente cliente = new Cliente(nombre, dni, telefono);
    return new Cuenta(cliente);
} SE CREA UNA NUEVA CUENTA
PIDIENDO LOS DATOS DEL
CLIENTE
DEVUELVE LA CUENTA NUEVA
```

Esto solo se hace si cuenta.dat no existe, y con estos datos se crea una nueva cuenta
Si el fichero cuenta.dat ya existe, se utiliza ObjectInputStream para recuperar la cuenta que estaba guardada. Así se mantienen los datos y movimientos realizados en ejecuciones anteriores

Óscar Martino Mora 2ºDAM Getafe

Y después se muestra el menú

```
public static void mostrarMenu(Scanner teclado, Cuenta cuenta) { 1usage
    int opcion; INICIALIZA LA VARIABLE OPCION
    do{
        System.out.println("Elige una opción: ");
        System.out.println("1. Ingresar dinero"); MUESTRA OPCIONES
        System.out.println("2. Retirar dinero");
        System.out.println("3. Consultar cuenta");
        System.out.println("4. Exportar movimientos");
        System.out.println("5. Salir");
        opcion = leerEntero(teclado, mensaje: "Elige una opción: "); PREGUNTA OPCION Y LA LEE MEDIANTE FUNCION EXTERNA PARA VERIFICAR QUE ES UN ENTERO
        switch (opcion){ VER QUE HACER SEGUN LA OPCION
            case 1:
                ingresar(teclado, cuenta); INGRESAR CON FUNCION EXTERNA QUE USA EL INGRESAR DE CUENTA
                break;
            case 2:
                retirar(teclado, cuenta); RETIRAR CON FUNCION EXTERNA QUE USA EL RETIRAR DE CUENTA
                break;
            case 3:
                cuenta.mostrarInformacion(); MOSTRAR LA INFORMACION CON EL MÉTODO DE CUENTA
                break;
            case 4:
                exportarCSV(cuenta); EXPORTAR LOS MOVIMIENTOS A UN ARCHIVO CSV
                break;
            case 5:
                System.out.println("Saliendo..."); SALIR
                break;
            default:
                System.out.println("opción no válida"); POR SI SE METE UNA OPCION QUE NO SEA NI 1, 2 3, 4 O 5
        }
    }while (opcion!=5);
}
```

Y luego se guarda la cuenta, después de terminar todas las operaciones realizadas en el menú

```
public static void guardarCuenta(Cuenta cuenta) throws IOException { 1usage
    ObjectOutputStream salida =new ObjectOutputStream( new FileOutputStream(FICHERO_CUENTA) );
    salida.writeObject(cuenta); guarda el objeto cuenta dentro del fichero
    salida.close(); cierra el fichero
}
```

```
ObjectOutputStream salida = new ObjectOutputStream(
new FileOutputStream(FICHERO_CUENTA) );
```

abre el fichero para escribir en él. Si el fichero no existe lo crea
permite guardar objetos Java dentro del fichero

```
public static int leerEntero(Scanner teclado, String mensaje) { 1usage
    while (true){ BUCLE      INTRODUCE USUARIO      MENSAJE "PREDETERMINADO", POR EJEMPLO EL DE "ELIGE UNA OPCION"
        try {
            System.out.print(mensaje); MUESTRA EL MENSAJE
            return Integer.parseInt(teclado.nextLine()); Y DEVUELVE EL ENTERO QUE METE EL USUARIO, "2" PASA A 2 CON EL
        } catch (NumberFormatException e) { SI NO ES UN NÚMERO "DA ERROR", PORQUE "HOLA" NO SE PUEDE "CONVERTIR"
            System.out.println("Introduce un número válido.");
        }
    }
}
```

se utiliza para leer un número entero introducido por el usuario. Utiliza Integer.parseInt() para convertir el texto introducido en un número y controla los errores mediante NumberFormatException . Si el usuario mete un valor que no es un número válido se muestra un mensaje y se vuelve a pedir el dato

Exportación de movimientos:

En el caso de elegir la opción 4, se exportan los movimientos de la cuenta a un archivo CSV, y he elegido este formato porque es fácil de utilizar para guardar información en filas y columnas y, además, luego se puede exportar fácilmente a Excel para verlo "más claro"

```
public static void exportarCSV(Cuenta cuenta) { 1usage
    DateTimeFormatter formato =DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss"); DAR FORMATO A LA FECHA
    try {
        BufferedWriter escritor =new BufferedWriter(new OutputStreamWriter(new FileOutputStream(FICHERO_CSV),StandardCharsets.UTF_8 ));
        escritor.write( str: "Fecha,Tipo,Concepto,Importe,Saldo resultante"); "CABECERA DEL ARCHIVO"
        escritor.newLine(); SALTO DE LINEA
        for (Movimiento movimiento :cuenta.getMovimientos()) {
            escritor.write( str: "\""+movimiento.getFecha().format(formato)+"\", \""+escaparCSV(movimiento.getTipo()) + BUCLE FOR PARA ESCRIBIR TODOS
                "\", \""+escaparCSV(movimiento.getConcepto())+"\", \""+String.format("%.2f",movimiento.getImporte()) + LOS MOVIMIENTOS
                "\", \""+String.format("%.2f",movimiento.getSaldoRestante()) );
            escritor.newLine(); SALTO DE LINEA
        }
        escritor.close(); CERRAR FICHERO
        System.out.println("movimientos exportados correctamente");
        System.out.println("Fichero: " + FICHERO_CSV);
    } catch (IOException e) {
        System.out.println("Error al exportar: "+e.getMessage());
    }
}
```

```
BufferedWriter escritor = new BufferedWriter( new OutputStreamWriter( new
FileOutputStream(FICHERO_CSV), StandardCharsets.UTF_8 ) );
```

Esto abre/crea el archivo (depende si existe o no) para escribir con el BufferedWriter que permite escribir en el fichero de forma cómoda

escaparCSV() sirve para preparar los textos para el formato CSV, por si contienen comas u otros caracteres especiales

```
public static String escaparCSV(String texto) { 2usages
    return texto.replace( target: "\", replacement: "\"");
}
```

Sustituye cada comilla por dos comillas, evitando problemas cuando un concepto o tipo de movimiento contiene comillas

ingresar() y retirar() gestionan la entrada de datos desde el teclado y llaman a los métodos de Cuenta

```
public static void ingresar(Scanner teclado, Cuenta cuenta) { 1 usage
    try {
        double importe = leerDouble(teclado, mensaje: "Importe a ingresar: ");
        System.out.print("Concepto: ");
        String concepto = teclado.nextLine();
        cuenta.ingresar(importe, concepto);
        System.out.printf("Ingreso realizado. Saldo: %.2f €\n", cuenta.getSaldo());
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

SU USA LA CUENTA DEL ARCHIVO cuenta.dat

PREGUNTA LA CANTIDAD A INGRESAR MEDIANTE LA FUNCION DE LEERDOUBLE() PARA VERIFICAR QUE SE META UN NUMERO (CON O SIN DECIMALES)

SE PREGUNTA E INTRODUCE EL CONCEPTO

SE USA EL MÉTODO INGRESAR DE LA CLASE CUENTA

SE MUESTRA EL SALDO RESTANTE DE LA CUENTA CON EL METODO GETSALDO DE LA CLASE CUENTA

CONTROL DE EXCEPCIONES

```
public static void retirar(Scanner teclado, Cuenta cuenta) { 1 usage
    try {
        double importe = leerDouble(teclado, mensaje: "Importe a retirar: ");
        System.out.print("Concepto: ");
        String concepto = teclado.nextLine();
        cuenta.retirar(importe, concepto);
        System.out.printf("Retirada realizada. Saldo: %.2f €\n", cuenta.getSaldo());
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

SE USA LA CUENTA DE cuenta.dat

SE PREGUNTA LA CANTIDAD A RETIRAR CON EL METODO LEERDOUBLE() PARA VERIFICAR

PIDE EL CONCEPTO

SE USA EL MÉTODO RETIRAR() DE LA CLASE CUENTA

SE MUESTRA EL SALDO RESTANTE DE LA CUENTA CON EL METODO GETSALDO DE LA CLASE CUENTA

CONTROL DE EXCEPCIONES

%.2f significa que muestra el número con 2 decimales

```
public void ingresar(double importe, String concepto) { 1 usage
    if (importe <= 0) {
        throw new IllegalArgumentException("Importe invalido, debe ser mayor de 0");
    }
    saldo = saldo + importe;
    Movimiento movimiento = new Movimiento("INGRESO", concepto, importe, saldo);
    movimientos.add(movimiento);
}

/**
 * realiza una retirada de dinero
 *
 * @param importe cantidad retirada
 * @param concepto concepto de la retirada
 */
public void retirar(double importe, String concepto) { 1 usage
    if (importe <= 0) {
        throw new IllegalArgumentException("Importe invalido, debe ser mayor de 0");
    } else if (importe > saldo) {
        throw new IllegalArgumentException("Importe invalido, debe ser menor que el saldo");
    } else {
        saldo = saldo - importe;
        Movimiento movimiento = new Movimiento("RETIRADA", concepto, importe, saldo);
        movimientos.add(movimiento);
    }
}
```

VER QUE LA CANTIDAD SEA MAYOR A 0 (VALIDA)

METODO INGRESAR DE LA CLASE CUENTA

SI ES VALIDA, SE SUMA AL SALDO Y SE CREA Y SE AÑADE UN NUEVO MOVIMIENTO

METODO RETIRAR DE LA CLASE CUENTA

VER QUE LA CANTIDAD A RETIRAR SEA VALIDA (MAYOR QUE 0 Y MENOR AL SALDO DISPONIBLE)

SI ES VALIDA SE RESTA AL SALDO Y SE CREA/AÑADE UN NUEVO MOVIMIENTO

Para mostrar la información de la cuenta (en la opción 3)

```
public void mostrarInformacion(){ 1 usage
    System.out.println("Cuenta: ");
    System.out.println(cliente);
    System.out.printf("Saldo: "+saldo);
    System.out.println();
    System.out.println("Movimientos: ");
    if (movimientos.isEmpty()){
        System.out.println("No hay movimientos");
    }else{
        for (Movimiento movimiento : movimientos) {
            System.out.println(movimiento);
        }
    }
}
```

SE MUESTRA LA INFORMACIÓN DEL CLIENTE Y SU SALDO

Y SE MUESTRAN LOS MOVIMIENTOS SI LA COLECCIÓN ESTA VACÍA, MUESTRA QUE NO HAY MOVIMIENTOS

Y SI NO ESTA VACÍA, RECORRE EL ARRAY Y MUESTRA LOS MOVIMIENTOS

Justificación del formato elegido para la exportación

Para exportar los movimientos he elegido el formato CSV porque es sencillo y permite guardar los datos separados por comas, por lo que puede abrirse fácilmente con programas como Microsoft Excel

Además, cada movimiento ocupa una línea y los datos están organizados en diferentes columnas: Fecha, Tipo, Concepto, Importe, Saldo resultante

He utilizado UTF-8 para la escritura del fichero, para poder trabajar bien con caracteres como la ñ y los acentos

Pruebas (videos)

Video de la creación de documentación JavaDoc:

https://drive.google.com/file/d/1Lv_9H7eJoXzqeX8tjARaQZPYRaIEZnoA/view?usp=sharing

Video ejecución programa: <https://drive.google.com/file/d/10bmgzOaTH8v-znszeHWHd9CijtiXmBCp/view?usp=sharing>

En el video de la ejecución se ve que al principio no están ninguna de las carpetas ni archivos creados

Y la documentación JavaDoc está dentro del proyecto

